

VOCABULARY — ENGLISH + TELUGU

Programming Terms The Plain Dictionary

Every word you'll meet while coding — function, class, method, built-in, custom, hook, API, and 80 more — each with a simple meaning, a tiny example, and a Telugu explanation.

"You don't truly know a concept until you can name it. Half of feeling lost as a beginner is just not knowing what the words mean. This fixes that half."

90+

TERMS

10

GROUPS

4

DIAGRAMS

How to use this dictionary

This is a **reference**, not a read-once book. When a word trips you up — in a tutorial, a job description, a Stack Overflow answer, or your own head — look it up here. Each term gives you: a **plain meaning**, a tiny **example**, and a **Telugu** line. Deep concepts (closures, the event loop, prototypes) get a short definition here and a pointer to the full Phase 4 & 5 notes.

Telugu: Idi okkasari chadive book kaadu — **reference**. E word ayina ardhham kakapothe — tutorial lo, job description lo, nee burralo — ikkada vethiki chudu. Prathi term ki: **simple meaning**, chinna **example**, oka **Telugu** line. Peddha concepts (closures, event loop) ki chinna definition + Phase 4-5 notes ki pointer.

Table of Contents

- [Part A — The Atoms of Code](#) · variable, value, statement, expression, syntax...
- [Part B — Functions & Their Family](#) · function, parameter, argument, method, built-in, custom, callback, hook...
- [Part C — Objects, Classes & OOP](#) · object, property, class, instance, constructor, inheritance...
- [Part D — Data & Collections](#) · data type, array, index, object-as-map, JSON...
- [Part E — Making Decisions & Repeating](#) · boolean, condition, loop, iteration...
- [Part F — Organising & Reusing Code](#) · module, package, library, framework, API, dependency, npm...
- [Part G — The Web & React Vocabulary](#) · DOM, event, component, props, state, hook, JSX, frontend/backend...
- [Part H — Running Code & When It Breaks](#) · runtime, scope, error, bug, debugging, sync/async...
- [Part I — The Developer's Workbench](#) · IDE, terminal, Git, repo, commit, deploy, environment...
- [Part J — The Confusables](#) · the pairs everyone mixes up, side by side

Part A — The Atoms of Code

The smallest words. Everything else is built from these.

Syntax — the grammar rules of a language: where the brackets, semicolons and quotes go. Break the syntax and the code won't even run.

Telugu: Bhasha yokka **grammar niyamalu** — brackets, semicolons ekkada raavaalo. Syntax tappithe code run ye avvadu.

Statement — one complete instruction that *does* something. Usually one line, often ending in `;`. `let total = 0;` and `if (x > 5) { ... }` are statements.

Telugu: Oka **pani chese** poorthi aagna — saadhaaranam ga oka line.

Expression — any piece of code that *produces a value*. `2 + 2`, `price * qty`, `age >= 18` are all expressions. The rough test: if it can sit on the right of `=`, it's an expression. A statement *does*; an expression *is worth* something.

Telugu: Oka **value ni istundi**. Test: `=` ki kudivaipu pettagaligite adi expression. Statement *panichestundi*; expression *value istundi*.

Variable — a named box that holds a value you can use and change later. `let score = 10;` — `score` is the box, `10` is what's inside.

Telugu: Value ni pettukune **peru unna dabba**. Taruvata vaadukovachu, maarchukovachu.

Constant — a variable whose value can't be *reassigned* after it's set. Written with `const`.

Telugu: Set chesaka **malli maarchalemu** — `const` tho raastam.

Value — the actual data itself: `42`, `"Nikhil"`, `true`, a list, an object. The *thing* stored in a variable.

Telugu: Asalu **data** — dabba lopala unna vishayam.

Literal — a fixed value written directly in the code, exactly as it is. `42` (number literal), `"hi"` (string literal), `[1, 2]` (array literal), `{ a: 1 }` (object literal).

Telugu: Code lo **neruga raasina** value — `42`, `"hi"`, `[1,2]`.

Operator — a symbol that performs an action on values: `+` `-` `*` `/` `=` `===` `&&` `!`. The values it works on are its **operands**. In `price + tax`, the `+` is the operator; `price` and `tax` are operands.

Telugu: Pani chese **gurthu** (`+` `-` `===` `&&`). Adi pani chese values ni **operands** antaru.

Keyword (*reserved word*) — a word the language owns and gives special meaning: `if`, `return`, `const`, `function`, `class`. You can't use them as your own names.

Telugu: Language ki **sontha** prathyeka padaalu (`if`, `return`, `const`). Vaatini nee variable perlu ga vaadaleru.

Comment — a note for humans that the computer ignores. `// single line` or `/* multi-line */`.

Telugu: Manushula kosam raase note — computer **paticchukodu**. Comment code enduku ela raasavo cheptundi.

Declaration vs assignment — declaring *introduces* a variable (`let x;`); assigning *gives it a value* (`x = 5`). Doing both at once is **initialization** (`let x = 5;`).

Telugu: **Declaration** = variable ni parichayam cheyyadam (`let x`). **Assignment** = daaniki value ivvadam (`x = 5`). Rendu okkasari ante **initialization**.

Data type — the *kind* of a value: text (`string`), a number (`number`), yes/no (`boolean`), a list (`array`), a record (`object`). See Part D and the notes (B2).

Telugu: Value **e rakam** di — text, number, true/false, list, object.

Part B — Functions & Their Family

The most important word in programming, and all its relatives. If you learn one part, learn this one.

The core

Function — a reusable, named block of code that takes **inputs**, does a job, and usually **returns** an output. Write it once, use it anywhere.

```
function add(a, b) {
  return a + b;    // takes two inputs, returns their sum
}
```

Telugu: Malli malli vaadagalige **code mukka** — input istav, oka pani chesi, output istundi. **Okkasari** raasi, ekkadaina vaadukovachu.

Call (*invoke / run / execute*) — to actually *use* a function by writing its name with `()`. Defining a function doesn't run it; **calling** it does. `add(2, 3)` calls `add` and gets back `5`.

Telugu: Function ni `()` tho **vaadadam**. Function raayadam valla adi run avvadu — **call** chesthene run avutundi.

Parameter vs argument — the two words for "function inputs", and they're *not* the same. A **parameter** is the placeholder in the definition; an **argument** is the real value you pass when calling.

```
function greet(name) {      // `name` is the PARAMETER (the placeholder)
  return `Hi ${name}`;
}
greet("Nikhil");           // "Nikhil" is the ARGUMENT (the real value)
```

Telugu: **Parameter** = definition lo unna *khaali chotu* (`name`). **Argument** = call chesinappudu pampe *nijamaina value* (`"Nikhil"`). Rendu veru!

Return — the value a function hands back to whoever called it. After `return`, the function stops. No `return` → the function gives back `undefined`.

Telugu: Function **tirigi ichhe** value. `return` taruvata function aagipotundi. `return` lekapothe `undefined` istundi.

Body — the code inside a function's `{ }` — the actual work it does.

Telugu: Function `{ }` lopali code — adi chese asalu pani.

Here is the whole anatomy in one picture:

keyword	name	parameters	
function	add	(a , b)	{
		return a + b ;	← body (the work) + return value
			}
	add(2 , 3)	← a CALL, with 2 and 3 as arguments	
	└──┘		
	arguments		

Kinds of functions

Built-in function (*pre-built / native*) — a function the language or browser **already gives you**. You didn't write it; you just use it. `console.log()`, `Math.max()`, `Array.map()`, `fetch()`, `parseInt()` are all built-in.

Telugu: Language leda browser **munde ichhina** function. Nuvvu raayaledu — vaadukuntav matrame (`console.log` , `Math.max` , `fetch`).

Custom function (*user-defined*) — a function **you write yourself** for your own needs.

```
function calculateGST(amount) { // YOUR function - doesn't exist until you write it
  return amount * 0.18;
}
```

Telugu: Nee avasaraaniki **nuvvu raase** function. Nuvvu raasedaaka adi undadu.

Method — a function that **belongs to an object**, called with a dot. Every method is a function; not every function is a method. `cart.push(item)` — `push` is a method of the array `cart` . `"hi".toUpperCase()` — `toUpperCase` is a method of the string.

Telugu: Oka **object ki chendina** function — dot tho pilustam. **Prathi method oka function**; kaani prathi function method kaadu. Dot ki mundu unna daaniki adi chendindi.

Arrow function — a shorter way to write a function, using `=>`. Its special trait: it has no `this` of its own (notes G1). `const double = (n) => n * 2;`

Telugu: `=>` tho raase **chinna** function. Pratyekata: daaniki sonta `this` undadu.

Anonymous function — a function with no name, usually made to be used once and passed straight into something else. `setTimeout(() => alert("Hi"), 1000);` — that arrow is anonymous.

Telugu: **Peru leni** function — okkasari vaadadaaniki, neruga maro function loki pamputam.

Callback — a function you **pass to another function**, to be called *later* — when something finishes or an event happens. The heartbeat of JavaScript.

```
button.addEventListener("click", () => console.log("clicked!"));
//                               |————— this is the callback —————|
```

Telugu: Maro function ki **pampe** function — *taruvata* (pani aipoyaka, leda event jarigaka) pilavadaniki. JavaScript gunde chappudu ide.

Higher-order function — a function that **takes a function as input, or returns one**. `map` , `filter` , `addEventListener` are all higher-order.

Telugu: Maro **function ni teeskune**, leda **function ni ichhe** function (`map` , `filter`).

Pure function — a function that, for the same input, always gives the same output and changes nothing outside itself. The easiest kind to test and trust (notes L7).

Telugu: Oke input ki **eppudu oke output**, bayata emi maarchadu. Test cheyyadaniki, nammadaniki sulabham.

Recursion — a function that **calls itself** to solve a problem by shrinking it each time. Needs a "stop" case or it loops forever.

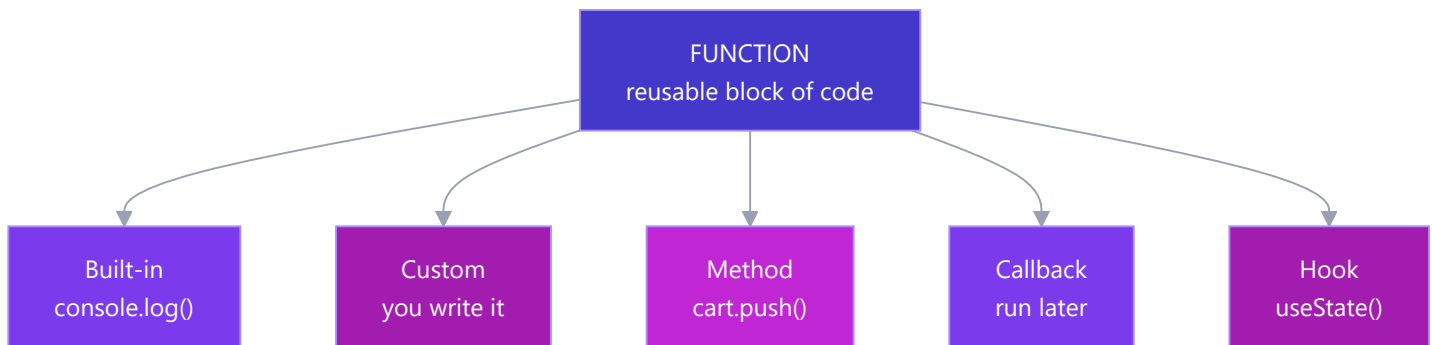
```
function countdown(n) {
  if (n === 0) return;    // the stop case - without it, forever
  console.log(n);
  countdown(n - 1);      // calls ITSELF with a smaller number
}
```

Telugu: **Tananu tanu piliche** function — prathisari problem ni chinna cheskuntundi. Oka "aagu" case tappanisari, lekapothe anantam ga tirugutundi.

Hook — in general, a **spot a framework lets you "plug into"** to run your own code. In **React** (your Phase 6), a hook is a special built-in function whose name starts with **use** — it lets a component *hook into* React features like memory and lifecycle. **useState()** gives a component memory; **useEffect()** runs code after render.

Telugu: Framework nee code ni **plug cheyanichhe chotu**. **React** lo, hook ante **use** tho modalayye special function — component ki state (**useState**), lifecycle (**useEffect**) laanti features ni *hook* chestundi. (Phase 6 lo vastundi.)

Where these fit together:



Part C — Objects, Classes & OOP

The words for modelling real-world "things" in code.

Object — a bundle of related data (and functions) kept together as **key: value** pairs. Models a "thing" — a user, a product, an order.

```
const user = { name: "Nikhil", age: 26, isAdmin: false };
```

Telugu: Sambandhamunna data ni kalipi unche **mukka** — **key: value** jantalu. Oka "vastuvu" ni chupistundi (user, product).

Property — one **key: value** pair inside an object. **name: "Nikhil"** is a property; **name** is its **key**, **"Nikhil"** is its value. Read it with a dot: **user.name**.

Telugu: Object lopali **oka key: value** janta. **name** = key, **"Nikhil"** = value. Dot tho chaduvu: **user.name**.

Method (*recap*) — a property whose value is a function. It's the object's *behaviour*, as opposed to its *data*. **user.greet()** — a method. **user.name** — a data property.

Telugu: Value function ayina property. Adi object yokka **pani** (data kaadu).

Class — a **blueprint** for making objects of the same kind. It defines what data and methods every object of that type will have.

```
class User {  
  constructor(name) { this.name = name; }  
  greet() { return `Hi, ${this.name}`; }  
}
```

Telugu: Oke rakam objects thayaru cheyyadaniki **plan (blueprint)**. Aa type prathi object ki e data, e methods untayo class cheptundi.

Instance (object) — one actual object built from a class. The class is the blueprint; the instance is the built house.

`const u = new User("Nikhil");` — `u` is an instance of `User`.

Telugu: Class nunchi thayarayina **oka nijamaina object**. Class = plan, instance = **kattina illu**.

new — the keyword that creates a fresh instance from a class and runs its constructor.

Telugu: Class nunchi **kotha instance** ni thayaru chese keyword.

Constructor — the special method that runs **once**, automatically, when an instance is created — it sets up the starting data.

Telugu: Instance thayarayinappudu **okkasari** automatic ga run ayye method — modati data ni set chestundi.

this — inside a method, a word meaning "the current object I'm working on". `this.name` = *this* particular user's name (notes G1).

Telugu: Method lopala, "**ippudu nenu pani chestunna object**" ani artham. `this.name` = ee user peru.

Inheritance — one class **building on another**, getting all its data and methods, then adding more. Uses

`extends`. `class Admin extends User { ... }` — an Admin *is a* User, plus extra powers (notes G4).

Telugu: Oka class inko daani **meeda kattadam** — daani data/methods anni techukoni, extra cherustundi. `extends` vaadutam.

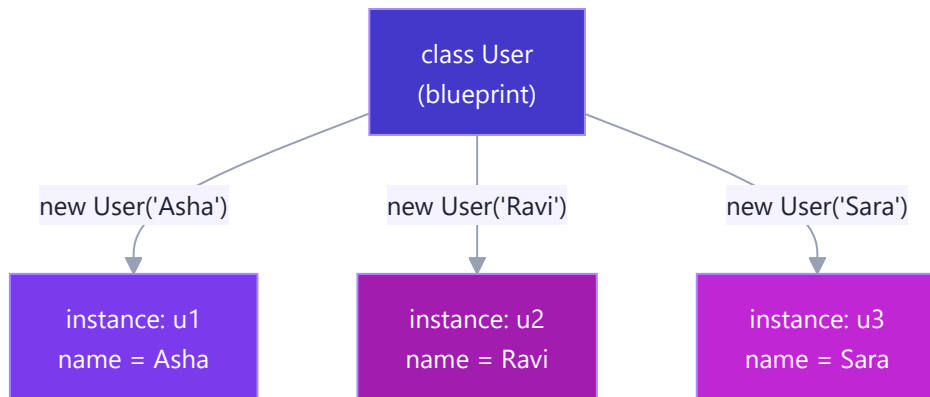
Encapsulation — **hiding** an object's internal data and only letting it change through safe methods. The `#` marks truly private fields (notes G5).

Telugu: Object lopali data ni **daachi**, safe methods dwara matrame maarchanivvadam. `#` = nijam ga private.

Polymorphism — different classes answering the **same method name** in their own way, so one line of code handles them all (notes G6).

Telugu: Veru veru classes **oke method peruki** tama sonta jawaabu ivvadam — okate line anni handle chestundi.

The class-to-instance relationship:



Part D — Data & Collections

The words for the shapes your data comes in.

Data type — the kind of a value. JavaScript's basics: `string` (text), `number`, `boolean` (true/false), `null`, `undefined`, plus `object` and `array` for structured data (notes B2).

Telugu: Value **e rakam** di — text, number, true/false, null, undefined, object, array.

String — text, written in quotes. `"Nikhil"`, `'hello'`, ``Hi ${name}``.

Telugu: Text — quotes lo raastam.

Number — any number; JS has no separate int/decimal. `42`, `3.14`, `-7`.

Telugu: Anke — JS lo int/decimal veru ledu.

Boolean — a value that is only `true` or `false`. The basis of every decision.

Telugu: `true` leda `false` matrame. Prathi nirnayaaniki adhaaram.

null vs undefined — both mean "no value", differently. `undefined` = JS's "never set". `null` = the developer's deliberate "empty on purpose".

Telugu: Rendu "value ledu" ye. `undefined` = JS "set cheyaledu". `null` = developer "kaavalane khaali".

Array — an **ordered list** of values, in `[ ]`. Positions counted from **0**. `const cart = ["pen", "book", "lamp"];`

Telugu: `[ ]` lo **vaparusa list**. Positions **0** nunchi.

Element — one item inside an array. In `["pen", "book"]`, `"pen"` is an element.

Telugu: Array lopali **oka item**.

Index — the position number of an element, starting at 0. `cart[0]` is the first element.

Telugu: Element yokka **position number**, 0 nunchi. `cart[0]` = modatidi.

Length — how many elements an array has. `cart.length`.

Telugu: Array lo **enni items** unnayo. `cart.length`.

Key — the name half of a `key: value` pair in an object. You look up values *by* their key.

Telugu: Object lo `key: value` lo **peru bhaagam**. Key tho value ni vethukutaam.

JSON (*JavaScript Object Notation*) — the universal **text format** for sending data between programs. Looks like a JS object but is a string. `JSON.stringify` turns an object into JSON text; `JSON.parse` turns it back (notes D6).

Telugu: Programs madhya data pampe **universal text format**. JS object laage kanipistundi kaani adi **string**. Bayataki `stringify`, lopaliki `parse`.

Mutable vs immutable — mutable data **can be changed** in place; immutable data **cannot** — you make a new copy instead. Arrays and objects are mutable; strings and numbers are immutable.

Telugu: **Mutable** = maarchagalige; **immutable** = maarchaleni (kotha copy chestham). Arrays/objects mutable; strings/numbers immutable.

Part E — Making Decisions & Repeating

The words for "do this only if..." and "do this again and again."

Condition — an expression that is either true or false, used to decide what runs. `age >= 18`, `cart.length === 0`.

Telugu: True leda false aye expression — em run avalo **nirnayinchadaniki**. `age >= 18`.

if / else — run one block when a condition is true, another when it's false. The most basic decision.

Telugu: Condition nijamaithe oka block, kakapothe inko block. Atyanta moolika nirnayam.

Loop — a block of code that **repeats**, usually until a condition stops it. `for`, `while`, `forEach`.

Telugu: **Malli malli** jarige code block — condition aagedaaka.

Iteration — **one single pass** through a loop. A loop that runs 5 times does 5 iterations.

Telugu: Loop yokka **oka round**. 5 saarlu tirigite 5 iterations.

Iterable — anything you can loop over with `for...of`: arrays, strings, Sets, Maps.

Telugu: `for...of` tho tippagalige edaina — arrays, strings, Sets.

Truthy / falsy — how non-boolean values behave inside an `if`. Six values are **falsy** (`false`, `0`, `""`, `null`, `undefined`, `NaN`); everything else is **truthy** (notes B3).

Telugu: `if` lopala boolean kaani values ela pravartistayo. **Aaru** falsy; migilinaivi anni truthy.

break / continue — inside a loop, `break` **stops** it completely; `continue` **skips** to the next iteration.

Telugu: Loop lo `break` = motham **aapeyi**; `continue` = ee round **vadili** next ki.

Part F — Organising & Reusing Code

How real projects are split up, and how they borrow other people's code.

Module — a **single file** of code that shares some of its contents (`export`) and uses things from other files (`import`). One file = one module (notes I1).

Telugu: Oka **file** code — konni vishayaalu `export` chesi, veeru files nunchi `import` cheskuntundi. Oka file = oka module.

Import / export — `export` makes something in a file available to others; `import` pulls it in. This is how files share code.

Telugu: `export` = ee file lodi veeriki andubaatu lo pettadam; `import` = daanni techukovadam.

Package — a **bundle of reusable code** (usually a library) that you download and install into your project, with a name and version.

Telugu: Download chesi install cheskune **reusable code mutta** — peru mariyu version tho.

Library — a **collection of ready-made functions** you call to save time. **You** stay in charge and call *it*. Examples: lodash (utilities), axios (HTTP), day.js (dates).

Telugu: Time save cheyyadaniki **ready-made functions samahaaram**. **Nuvvu** control lo unti, daanini *nuvvu* pilustav.

Framework — a **complete structure** you build your app inside. It's in charge and **calls your code** at the right moments. The famous line: "*you call a library; a framework calls you.*" Examples: React, Angular, Next.js, Express.

Telugu: Nee app ni daani **lopala katte poorthi nirmaanam**. Adi control lo untundi, saraina samayamlo **nee code ni adi pilustundi**. "Library ni nuvvu pilustav; framework ninnu pilustundi."

Dependency — any package your project **needs to run**. Listed in `package.json`.

Telugu: Nee project run avadaniki **avasaramaina** package. `package.json` lo untundi.

Package manager (*npm, yarn, pnpm*) — the tool that downloads, installs, and tracks your packages. `npm install axios` fetches a package and its dependencies.

Telugu: Packages ni download, install, track chese **tool**. `npm install axios` — package ni techipedutundi.

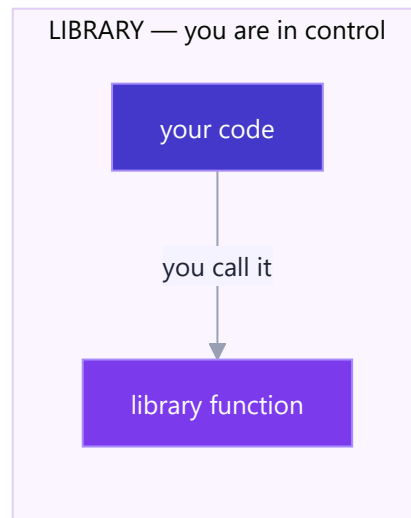
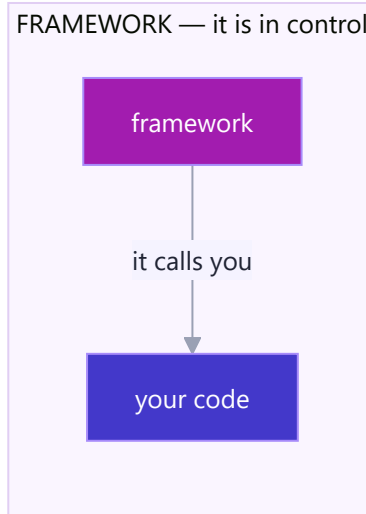
API (*Application Programming Interface*) — a **defined way for two pieces of software to talk**. Two common meanings: a **web API** (URLs you send requests to, get JSON back) and a **code API** (the set of functions a library exposes for you to use).

Telugu: Rendu software mukkalu **maatladukune padhati**. Rendu arthaalu: **web API** (URLs ki request pampi JSON techukovadam) mariyu **code API** (library ichhe functions samahaaram).

SDK (*Software Development Kit*) — a bigger toolbox than a library: packages, tools, and docs bundled to build for a specific platform (e.g. the "SAP AI SDK", the "AWS SDK").

Telugu: Library kanna **peddha toolbox** — oka platform kosam packages + tools + docs kalipi.

Library vs framework, the control flip:



Part G — The Web & React Vocabulary

The words you'll live in once you build for the browser — and the React terms coming in Phase 6.

Frontend vs backend — **frontend** is everything that runs in the user's **browser** (the buttons, the layout, what they see). **Backend** is everything that runs on the **server** (the database, the logic, the secrets).

Telugu: **Frontend** = user **browser** lo nadichedi (buttons, layout, kanipinchedi). **Backend** = **server** lo nadichedi (database, logic, secrets).

Client vs server — the **client** *asks* (usually the browser); the **server** *answers* (a computer that holds data and logic). Client sends a **request**, server sends a **response**.

Telugu: **Client** = *adugutundi* (browser); **server** = *jawaabu istundi*. Client **request** pamputundi, server **response** istundi.

DOM (*Document Object Model*) — the live **tree of objects** the browser builds from your HTML. JavaScript edits this tree and the page updates (notes E1).

Telugu: Browser nee HTML nunchi thayaru chese **objects chettu**. JS ee chettu ni maarusthundi, page update avutundf.

Element vs node — a **node** is any point in the DOM tree (including text and comments); an **element** is a node that's an actual HTML tag (`<div>` , `<p>`).

Telugu: **Node** = DOM chettu lo *edaina point* (text kuda). **Element** = HTML tag ayina node (`<div>`).

Event — **something that happens** on the page: a click, a keypress, a form submit, the page loading.

Telugu: Page meeda **emo jarigindi** — click, key press, form submit.

Event listener — a function that **waits for an event** and runs when it happens. Attached with `addEventListener` (notes E3).

Telugu: Event kosam **eduru chusi**, adi jarigganae run ayye function.

Component — a **reusable, self-contained piece of UI** — a button, a card, a navbar. React apps are built by combining components.

Telugu: Malli malli vaadagalige **UI mukka** — button, card, navbar. React apps ivi kalipi kadataru.

Props — the **inputs passed into a component** from its parent, read-only. Like arguments for a component.

`<Welcome name="Nikhil" />` — `name` is a prop.

Telugu: Parent nunchi component ki **pampe inputs**, chadavadaniki matrame. Component ki arguments laantivi.

State — data a component **owns and can change** over time; when state changes, the UI **re-renders** to match.

Telugu: Component **sontha ga unche, maarche** data. State maarite, UI **malli geeyabadutundi**.

Hook (React) — a built-in function starting with `use` that lets a component use React features. `useState` (memory), `useEffect` (run code after render). (Also in Part B.)

Telugu: `use` tho modalayye built-in function — component ki React features istundi. `useState` , `useEffect` .

JSX — HTML-like syntax written **inside JavaScript**, used by React. `const el = <h1>Hello {name}</h1>;`

Telugu: JavaScript **lopala** raase HTML laanti syntax — React vaadutundi.

Render — to **produce the visible UI** from your data/components. A **re-render** happens when state changes.

Telugu: Data nunchi **kanipinche UI ni thayaru** cheyyadam. State maarite **re-render**.

Responsive — a layout that **adapts** to any screen size, phone to desktop.

Telugu: E screen size ki ayina **saripoye** layout — phone nunchi desktop varaku.

Part H — Running Code & When It Breaks

What happens when code runs, and the words for when it doesn't.

Runtime — two meanings: (1) the **time when your program is running** (as opposed to when you're writing it); (2) the **environment that runs your code** — Node.js is a JavaScript runtime.

Telugu: Rendu arthaalu: (1) program **run avutunna samayam**; (2) code ni **run chese vaatavaranam** — Node.js oka JS runtime.

Compile vs interpret — **compiling** translates the whole program to machine code *before* running (C++).

Interpreting runs it line by line *as it goes* (classic JS). Modern JS mixes both for speed.

Telugu: **Compile** = motham program ni run ki *mundu* machine code loki maarchadam. **Interpret** = line by line *appatikappudu* run cheyyadam.

Scope — **where a variable is visible/usable** in your code. Global scope = everywhere; local scope = only inside a function or block (notes F3).

Telugu: Variable **ekkada kanipistundo/vaadocho**. Global = anta chota; local = oka function/block lopala matrame.

Closure — a function that **remembers the variables** from where it was created, even after that outer function has finished (notes F4).

Telugu: Tanu **puttina chota variables ni gurthu** pettukune function — bayati function aipoyaka kuda.

Synchronous vs asynchronous — **sync** code runs one line at a time, each waiting for the last. **Async** code can **start** a slow job (a network call) and **move on**, handling the result later (notes H).

Telugu: **Sync** = okati taruvata okati, prathidi eduru chustundi. **Async** = slow pani (network) **start chesi munduku**, result taruvata chuskuntundi.

Promise — an object standing for a value that **isn't ready yet** — a receipt for a future result. **async/await** is the modern way to use them (notes H4).

Telugu: **Inka ready kaani** value ki receipt — future result ki token laantidi.

Bug — a **mistake in the code** that makes it behave wrong. Named after a real moth found in a computer in 1947.

Telugu: Code ni tappuga panichese **porapaatu**.

Error / exception — the **signal JS raises when something goes wrong** at runtime (e.g. **TypeError**, **ReferenceError**). Caught with **try/catch** (notes J3).

Telugu: Emaina tappu jariginappudu JS ichhe **signal** (**TypeError**). **try/catch** tho pattukuntaam.

Stack trace — the **list of function calls** that led to an error, printed with it. Read top-down; the top is usually where it broke (notes J12).

Telugu: Error ki daari teesina **function calls list**. Paina nunchi chaduvu — paidi mostam ekkada pagilindo adi.

Debugging — the craft of **finding and fixing** bugs — with logs, breakpoints, and reasoning, not guessing (notes L / J12).

Telugu: Bugs ni **kanukkoni sariddi** cheye naipunyam — logs, breakpoints tho, oohatho kaadu.

Console — the panel (and the **console.log** function) where your messages and errors print. Your most-used debugging tool.

Telugu: Nee messages/errors kanipinche panel (mariyu **console.log**). Ekkuva vaade debugging tool.

Part I — The Developer's Workbench

The everyday tools and words around the code itself.

IDE / editor — the app you **write code in**. VS Code is the popular one; it adds autocomplete, error highlighting, and debugging on top of a plain text editor.

| **Telugu:** Code **raase app** — VS Code popular. Autocomplete, error highlight, debugging isthundi.

Terminal / CLI (*Command-Line Interface*) — the **text window** where you type commands to run tools (`npm install` , `git push` , `node app.js`).

| **Telugu:** Commands type chese **text window** — `npm install` , `git push` laanti tools ni run chestundi.

Git — the **version-control** tool that tracks every change to your code, so you can go back, branch, and collaborate.

| **Telugu:** Nee code loni prathi maarpu ni **track chese** tool — venakki velladaniki, branch cheyyadaniki, kalisi pani cheyyadaniki.

Repository (*repo*) — a **project folder that Git tracks**. On GitHub, it's your project's online home.

| **Telugu:** Git track chese **project folder**. GitHub lo, nee project online illu.

Commit — a **saved snapshot** of your changes, with a message describing them. Your project's undo history.

| **Telugu:** Nee maarpula **saved snapshot**, oka message tho. Project yokka undo charitra.

Branch — a **parallel line of work** where you can build a feature without touching the main code, then merge it back.

| **Telugu:** Main code ni muttukokunda feature katte **samaanantara daari** — taruvata merge chestham.

Push / pull — **push** sends your commits up to GitHub; **pull** brings others' commits down to you.

| **Telugu:** **Push** = nee commits ni GitHub ki paiki pampu; **pull** = veeri commits ni kindaki techuko.

package.json — the **manifest file** at a project's root: its name, scripts, and the list of dependencies.

| **Telugu:** Project root lo **vivaraala file** — peru, scripts, dependencies list.

node_modules — the (huge) folder where npm installs all your downloaded packages. Never edited by hand, never committed to Git.

| **Telugu:** npm packages anni install ayye (peddha) folder. Cheththo muttam, Git ki pampam.

Environment variable — a **config value kept outside the code** (API keys, database URLs), so secrets don't live in your files. Read via `process.env` .

| **Telugu:** Code **bayata unche config value** (API keys, DB URLs) — secrets files lo undakunda.

Build — the step that **turns your source code into runnable output** (bundling, minifying, transpiling) before you ship it.

| **Telugu:** Nee source code ni **run ayye output ga maarche** step — ship cheyyadaniki mundu.

Deploy — to **put your app on a live server** so real users can reach it. "Going to production."

| **Telugu:** Nee app ni **live server meeda pettadam** — nijamaina users vaadelaga. "Production loki velladam."

Transpile — to **convert code from one form to another**: TypeScript → JavaScript, or modern JS → older JS for old browsers (Babel does this).

| **Telugu:** Code ni **oka roopam nunchi inko daaniki maarchadam** — TypeScript → JS, modern JS → paatha JS.

Refactor — to **restructure code to make it cleaner** without changing what it does. Behaviour stays; readability improves.

Telugu: Code ni **clean cheyyadaniki** tirigi raayadam — pani okate untundi, chadavadam sulabham avutundi.

Part J — The Confusables (side by side)

The word-pairs that trip up every beginner, settled in one table. When two terms feel the same, come here.

These sound alike...	...but here's the difference
Parameter vs Argument	Parameter = placeholder in the <i>definition</i> (<code>function f(x)</code>). Argument = real value in the <i>call</i> (<code>f(5)</code>).
Function vs Method	A method is just a function that lives on an object and is called with a dot (<code>arr.push()</code>). All methods are functions.
Built-in vs Custom	Built-in = given by JS/browser (<code>fetch</code> , <code>Math.max</code>). Custom = you wrote it yourself.
Library vs Framework	You call a library; a framework calls you . Library = toolbox you control; framework = structure that runs your code.
Class vs Object	Class = the blueprint . Object (instance) = a real thing built from it. One class → many objects.
Package vs Module	Module = one file that imports/exports. Package = a downloadable bundle (often many modules) you install.
== vs ===	<code>==</code> converts types before comparing (surprises). <code>===</code> checks value and type. Always use <code>===</code> .
null vs undefined	<code>undefined</code> = JS never set it. <code>null</code> = you set it empty on purpose .
Statement vs Expression	Expression produces a value (<code>2+2</code>). Statement does an action (<code>if</code> , a loop).
Sync vs Async	Sync = one thing at a time, each waits. Async = start a slow job, carry on, handle the result later.
Compile vs Interpret	Compile = translate the whole program before running. Interpret = run it line by line as it goes.
Frontend vs Backend	Frontend = runs in the browser (what users see). Backend = runs on the server (data, logic, secrets).
Client vs Server	Client asks (browser sends a request). Server answers (sends a response).
Element vs Node	Node = any point in the DOM tree. Element = a node that's an actual HTML tag.
Props vs State	Props = inputs passed in from outside (read-only). State = data a component owns and changes .
Mutable vs Immutable	Mutable = can be changed in place (arrays, objects). Immutable = can't; you make a copy (strings, numbers).
HTML vs CSS vs JS	HTML = structure (what's on the page). CSS = style (how it looks). JS = behaviour (what it does).

These sound alike...

...but here's the difference

Bug vs Error

Bug = the mistake in your code. Error = the runtime signal that something went wrong because of it.

One-page memory card

Atoms: variable (named box) · value (the data) · expression (makes a value) · statement (does an action).
Function: reusable code; *parameter* = placeholder, *argument* = real value; *return* = what it hands back.
Function kinds: built-in (given) · custom (yours) · method (on an object) · callback (run later) · hook (plug into React). **OOP:** class (blueprint) → instance (built object) · property (data) · method (behaviour) · `this` (current object). **Data:** string · number · boolean · null/undefined · array (ordered list) · object (key:value) · JSON (data as text). **Reuse:** module (one file) · package (installed bundle) · library (you call it) · framework (calls you) · API (how software talks). **Web:** frontend (browser) vs backend (server) · client asks, server answers · DOM (page as a tree) · event + listener. **React:** component (UI piece) · props (inputs in) · state (owned, changing data) · hook (`useState`) · JSX · render. **Running:** runtime · scope (where a variable lives) · sync vs async · bug (mistake) → error (the signal) → debug (fix it). **Workbench:** IDE (VS Code) · terminal (commands) · Git (track changes) · repo · commit · push/pull · deploy (go live). **Golden rule:** when two words feel the same, open Part J — the difference is the whole point.

You now have the vocabulary. Every tutorial, every job description, every error message is written in these words — and now you speak them.